

字符串基本特点



羊肉串是羊肉的串
字符串是字符的串

- 1 字符串的本质是：字符序列。
- 2 Python不支持单字符类型，单字符也是作为一个字符串使用的。

#####

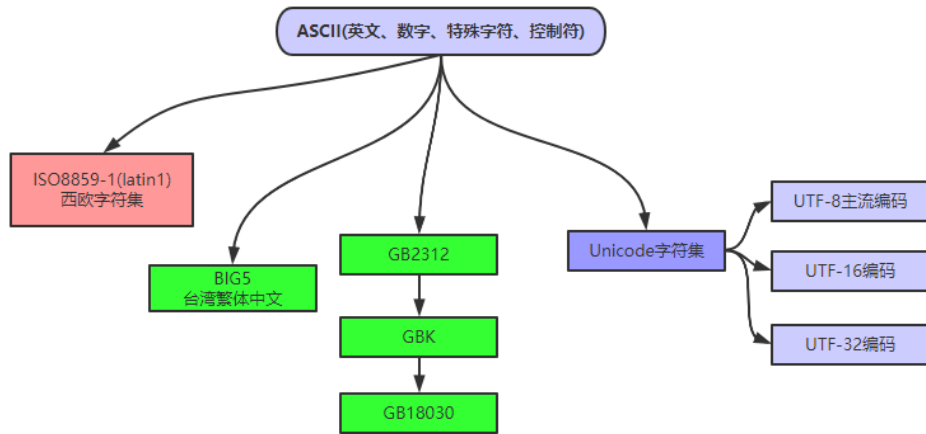
⚠ Python的字符串是不可变的，我们无法对原字符串做任何修改。但，可以将字符串的一部分复制到新创建的字符串，达到“看起来修改”的效果。

很多人初学编程时，总是担心自己数学不行，潜意识里认为数学好才能编程。实际上，大多数程序员打交道最多的是“字符串”而不是“数学”。因为，编程是用来解决现实问题的，因此逻辑思维的重要性远远超过数学能力。

字符串的编码

Python3直接支持Unicode，可以表示世界上任何书面语言的字符。Python3的字符默认就是16位Unicode编码，ASCII码是Unicode编码的子集。

字符集发展历史



使用内置函数ord()可以把字符转换成对应的Unicode码；

使用内置函数chr()可以把十进制数字转换成对应的字符。

```

1 >>> ord('A')           #65
2 >>> ord('高')           #39640
3 >>> chr(66)             #'B'
4 >>> ord('淇')           #28103
  
```

引号创建字符串

我们可以通过单引号或双引号创建字符串。例如：`a='abc'` `b="sxt"`

使用两种引号的好处是可以创建本身就包含引号的字符串，而不用使用转义字符。例如：

```

1 a = "I'm a teacher!"
2 print(a)                #I'm a teacher!
3 b = 'my_name is "TOM"'
4 print(b)                #my_name is "TOM"
  
```

连续三个单引号或三个双引号，可以帮助我们创建多行字符串。在长字符串中会保留原始的格式。例如：

```
1 s=''  
2     I  
3     Love  
4     Python  
5     ''  
6 print(s)
```

空字符串和len()函数

Python允许空字符串的存在，不包含任何字符且长度为0。例如：

```
1 c = ''  
2 print(len(c))      #结果： 0
```

len()用于计算字符串含有多少字符。例如：

```
1 d = 'abc尚学堂'  
2 len(d)      #结果： 6
```

实时效果反馈

1. python中字符串使用的是什么字符集？

- ☒ A ASCII
- ☐ B GBK
- ☐ C ISO8859-1

D Unicode

2. 如下代码，打印输出的结果是：

```
1 b = 'my_name is "TOM"'
2 print(b)
```

- A 会报错
- B my_name is "TOM"
- C my_name is TOM
- D 'my_name is "TOM"'

答案

1=>D 2=>B

转义字符

我们可以使用 `\+特殊字符`，实现某些难以用字符表示的效果。比如：换行等。常见的转义字符有这些：

转义字符	描述
(在行尾时)	续行符
\	反斜杠符号
'	单引号
"	双引号
\b	退格(Backspace)
\n	换行
\t	横向制表符
\r	回车

【操作】测试转义字符的使用

```
1 a = 'I\nlove\nU'  
2 print(a)  
3 print('aabb\\cc')
```

输出：

```
I  
love  
U  
aabb\\cc
```

字符串拼接

① 可以使用 `+` 将多个字符串拼接起来。例如： `'aa' + 'bb'` 结果是 `'aabb'`

- ① 如果 `+` 两边都是字符串，则拼接。
- ② 如果 `+` 两边都是数字，则加法运算
- ③ 如果 `+` 两边类型不同，则抛出异常

② 可以将多个字面字符串直接放到一起实现拼接。例如： `'aa' 'bb'` 结果是 `'aabb'`

【操作】字符串拼接操作

```
1 a = 'sxt'+'gaoqi'      #结果是: 'sxtgaoqi'  
2 b = 'sxt' 'gaoqi'     #结果是: 'sxtgaoqi'
```

字符串复制

使用*可以实现字符串复制

```
1 a = 'Sxt'*3  #结果:'SxtSxtSxt'
```

不换行打印

我们前面调用print时，会自动打印一个换行符。有时，我们不想换行，不想自动添加换行符。我们可以自己通过参数end = “任意字符串”。实现末尾添加任何内容：

```
1 print("sxt",end=' ')
2 print("sxt",end='##')
3 print("sxt")
```

运行结果：

sxt sxt##sxt

从控制台读取字符串

我们可以使用input()从控制台读取键盘输入的内容。

```
1 myname = input("请输入名字:")
2 print("您的名字是：" + myname)
```

执行结果：

请输入名字:高淇

您的名字是：高淇

实时效果反馈

1. 转义字符中，换行的哪一个？

- ☐ A `\r`
- ☐ B `\t`
- ☒ C `\n`
- ☐ D `\b`

2. 如下代码，输出正确结果是：

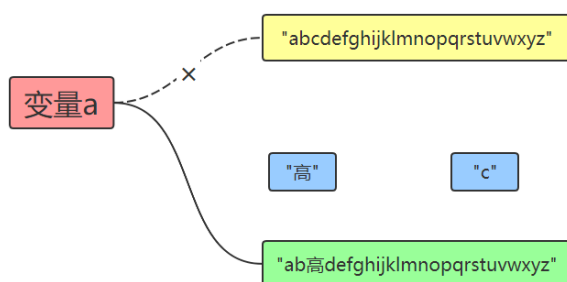
```
1 a = 13
2 b = "14"
3 c = a+b
4 print(c)
```

- ☐ A 会报错
- ☐ B 27
- ☒ C 1314
- ☐ D "27"

答案

1=>C 2=>A

replace() 实现字符串替换



字符串是“不可改变”的，我们通过[]可以获取字符串指定位置的字符，但是我们不能改变字符串。我们尝试改变字符串中某个字符，发现报错了：

```

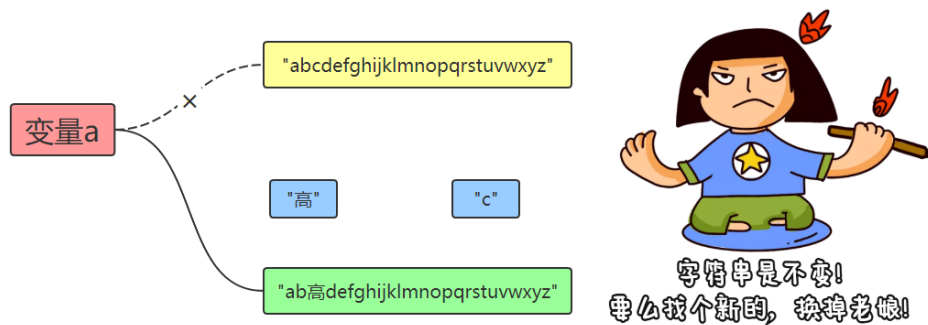
1 >>> a = 'abcdefghijklmnopqrstuvwxyz'
2 >>> a
3 'abcdefghijklmnopqrstuvwxyz'
4 >>> a[3]='高'
5 Traceback (most recent call last):
6   File "<pyshell#94>", line 1, in <module>
7     a[3]='高'
8 TypeError: 'str' object does not support item assignment

```


字符串不可改变。但是，我们确实有时候需要替换某些字符。这时，只能通过创建新的字符串来实现。

```
1 >>> a = 'abcdefghijklmnopqrstuvwxyz'
2 >>> a
3 'abcdefghijklmnopqrstuvwxyz'
4 >>> a = a.replace('c','高')
5 'ab高defghijklmnopqrstuvwxyz'
```

整个过程中，实际上我们是创建了新的字符串对象，并指向了变量a，而不是修改了以前的字符串。内存图如下：



str()实现数字转型字符串

str()可以帮助我们将其他数据类型转换为字符串。例如：

```
1 a = str(5.20)      #结果是: a = '5.20'
2 b = str(3.14e2)    #结果是: b = '314.0'
3 c = str(True)      #结果是: c = 'True'
```

♥当我们调用print()函数时，解释器自动调用了str()将非字符串的对象转成了字符串。

使用[]提取字符

字符串的本质就是字符序列，我们可以通过在字符串后面添加[]，在[]里面指定偏移量，可以提取该位置的单个字符。

① 正向搜索：

最左侧第一个字符，偏移量是0，第二个偏移量是1，以此类推。直到len(str)-1为止。

② 反向搜索：

最右侧第一个字符，偏移量是-1，倒数第二个偏移量是-2，以此类推，直到-len(str)为止。

【操作】使用[]提取字符串中的字符

```
1 >>> a = 'abcdefghijklmnopqrstuvwxyz'
2 >>> a
3 'abcdefghijklmnopqrstuvwxyz'
4 >>> a[0]
5 'a'
6 >>> a[3]
7 'd'
8 >>> a[26-1]
9 'z'
10 >>> a[-1]
11 'z'
12 >>> a[-26]
13 'a'
14 >>> a[-30]
```

```

15 | Traceback (most recent call last):
16 |     File "<pyshe11#91>", line 1, in <module>
17 |         a[-30]
18 | IndexError: string index out of range

```

实时效果反馈

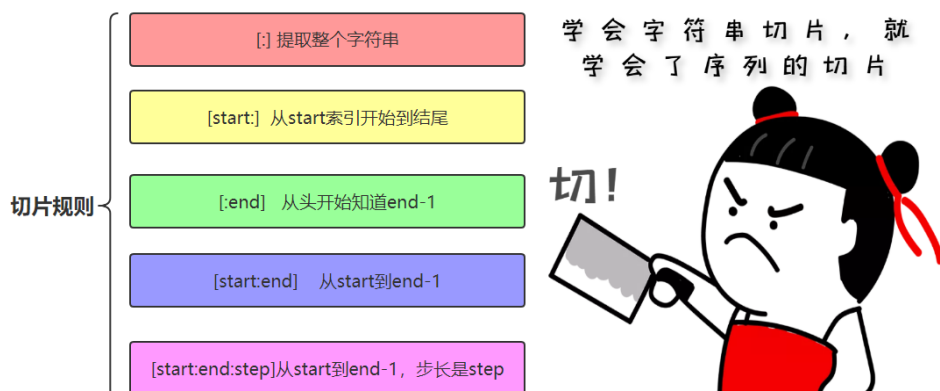
1. python中关于字符串的说法，错误的是：

- A** 字符串是内置类型
- B** 字符串不可改变
- C** 调用replace()改变的原字符串对象
- D** `c="abcdefg"` `c[-1]` 结果是: `g`

答案

1=>C

字符串切片slice操作



切片slice操作可以让我们快速的提取子字符串。标准格式为：

[起始偏移量start: 终止偏移量end: 步长step]

典型操作(三个量为正数的情况)如下：

操作和说明	示例	结果
[:] 提取整个字符串	"abcdef"[:]	"abcdef"
[start:] 从start索引开始到结尾	"abcdef"[2:]	"cdef"
[:end] 从头开始知道end-1	"abcdef"[:2]	"ab"
[start:end] 从start到end-1	"abcdef"[2:4]	"cd"
[start:end:step] 从start提取到end-1，步长是step	"abcdef"[1:5:2]	"bd"

其他操作（三个量为负数）的情况：

示例	说明	结果
"abcdefghijklmnpqrstuvwxy" [-3:]	倒数三个	"xyz"
"abcdefghijklmnpqrstuvwxy" [-8:-3]	倒数第八个到倒数第三个(包头不包尾)	'stuvw'
"abcdefghijklmnpqrstuvwxy" [::-1]	步长为负，从右到左反向提取	'zyxwvutsrqponmlkjihgfedcba'

切片操作时，起始偏移量和终止偏移量不在[0,字符串长度-1]这个范围，也不会报错。起始偏移量小于0则会当做0，终止偏移量大于“长度-1”会被当成-1。例如：

```
1 >>> "abcdefg"[3:50]
2 'defg'
```

我们发现正常输出了结果，没有报错。

实时效果反馈

1. 将"sxtsxtsxtsxtsxt"字符串中所有的s输出,如下的是:

A "sxtsxtsxtsxtsxt"[:-3]

B "sxtsxtsxtsxtsxt"[:-2]

C "sxtsxtsxtsxtsxt"[:-1]

D "sxtsxtsxtsxtsxt"[:-3]

2. 如下代码, 正确结果是:

```
1 "abcdefghijklmnopqrstuvwxyz"[::-1]
```

A 'zyxwvutsrqponmlkjihgfedcba'

B 'z'

C 'a'

D "abcdefghijklmnopqrstuvwxyz"

答案

1=>D 2=>A

split()分割和join()合并

split()可以基于指定分隔符将字符串分隔成多个子字符串(存储到列表中)。如果不指定分隔符, 则默认使用空白字符(换行符/空格/制表符)。示例代码如下:

```
1 >>> a = "to be or not to be"
2 >>> a.split()
3 ['to', 'be', 'or', 'not', 'to', 'be']
4 >>> a.split('be')
5 ['to ', ' or not to ', '']
```

join()的作用和split()作用刚好相反，用于将一系列子字符串连接起来。示例代码如下：

```
1 >>> a = ['sxt', 'sxt100', 'sxt200']
2 >>> '*'.join(a)
3 'sxt*sxt100*sxt200'
```

拼接字符串要点：

使用字符串拼接符 `+`，会生成新的字符串对象，因此不推荐使用 `+` 来拼接字符串。推荐使用 `join` 函数，因为 `join` 函数在拼接字符串之前会计算所有字符串的长度，然后逐一拷贝，仅新建一次对象。

【操作】测试+拼接符和join()，不同的效率 (mypy_07.py)

```
1 import time
2
3 time01 = time.time() #起始时刻
4 a = ""
5 for i in range(1000000):
6     a += "sxt"
7
```

```
8 time02 = time.time()    #终止时刻
9
10 print("运算时间: "+str(time02-time01))
11
12 time03 = time.time()    #起始时刻
13 li = []
14 for i in range(1000000):
15     li.append("sxt")
16
17 a = "".join(li)
18
19 time04 = time.time()    #终止时刻
20
21 print("运算时间: "+str(time04-time03))
```

实时效果反馈

1. `"to be or not to be".split()`，执行的结果是：

- A `"to be or not to be"`
- B `['to', 'be', 'or', 'not', 'to', 'be']`
- C `[to, be, or, not, to, be]`
- D `['to be', 'or', 'not to be']`

2. 使用 `+` 和 `join()` 拼接字符串，错误结果是：

- A 字符串拼接符 `+`，会生成新的字符串对象
- B `join` 函数仅新建一次对象
- C `+` 和 `join()` 的作用都是用来：拼接字符串

D + 和 `join()` 效率都是一样的，用哪个都无所谓

答案

1=>B 2=>D

字符串驻留机制和字符串比较

字符串驻留：常量字符串只保留一份。

```
1 c = "dd#"
2 d = "dd#"
3 print(c is d)      #True
```

字符串比较和同一性

我们可以直接使用 `==` `!=` 对字符串进行比较，是否含有相同的字符。

我们使用 `is` `not is`，判断两个对象是否同一个对象。比较的是对象的地址，即 `id(obj1)` 是否和 `id(obj2)` 相等。

成员操作符判断子字符串

`in` `not in` 关键字，判断某个字符(子字符串)是否存在于字符串中。

```
1 "ab" in "abcdefg"      #true
```

实时效果反馈

1. 如下代码，打印输出结果是：

```
1 a = "abd_33"  
2 b = "abd_33"  
3 print(a is b)  
4 print(a==b)
```

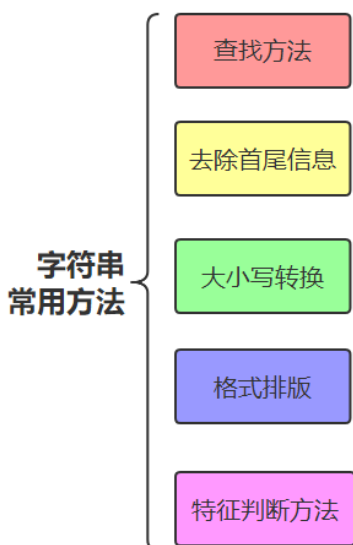
- A** ☐ True ☐ False
- B** ☐ True ☐ False
- C** ☐ True ☐ False
- D** ☐ True ☐ True

答案

1=>D

字符串常用方法汇总

字符串有很多常用的方法，我们需要熟悉。我们通过表格将这些方法汇总起来，方便大家查阅。希望大家针对每个方法都做一次测试。



方法很多！
好在名字都好记！

常用查找方法

我们以一段文本作为测试：

```
1 a='''我是高淇,我在北京尚学堂科技上班。我的儿子叫高洛
   希,他6岁了。我是一个编程教育的普及者,希望影响6000万
   学习编程的中国人。我儿子现在也开始学习编程,希望他18岁
   的时候可以超过我'''
```

方法和使用示例	说明	结果
len(a)	字符串长度	96
a.startswith('我是高淇')	以指定字符串开头	True
a.endswith('过我')	以指定字符串结尾	True
a.find('高')	第一次出现指定字符串的位置	2
a.rfind('高')	最后一次出现指定字符串的位置	29
a.count("编程")	指定字符串出现了几次	3
a.isalnum()	所有字符全是字母或数字	False

去除首尾信息

我们可以通过strip()去除字符串首尾指定信息。通过lstrip()去除字符串左边指定信息，rstrip()去除字符串右边指定信息。

【操作】去除字符串首尾信息

```

1 >>> "*s*x*t*".strip("*")
2 's*x*t'
3 >>> "*s*x*t*".lstrip("*")
4 's*x*t*'
5 >>> "*s*x*t*".rstrip("*")
6 '*s*x*t'
7 >>> "  s xt ".strip()
8 's xt'

```

大小写转换

编程中关于字符串大小写转换的情况，经常遇到。我们将相关方法汇总到这里。为了方便学习，先设定一个测试变量：

```
1 a = "gaoqi love programming, love SXT"
```

示例	说明	结果
a.capitalize()	产生新的字符串,首字母大写	'Gaoqi love programming, love sxt'
a.title()	产生新的字符串,每个单词都首字母大写	'Gaoqi Love Programming, Love Sxt'
a.upper()	产生新的字符串,所有字符全转成大写	'GAOQI LOVE PROGRAMMING, LOVE SXT'
a.lower()	产生新的字符串,所有字符全转成小写	'gaoqi love programming, love sxt'
a.swapcase()	产生新的,所有字母大小写转换	'GAOQI LOVE PROGRAMMING, LOVE sxt'

格式排版

`center()`、`ljust()`、`rjust()` 这三个函数用于对字符串实现排版。示例如下：

```
1 >>> a="SXT"
2 >>> a.center(10,"*")
3 '***SXT***'
4 >>> a.center(10)
5 '  SXT  '
6 >>> a.ljust(10,"*")
7 'SXT*****'
```

特征判断方法

- 1 isalnum() 是否为字母或数字
- 2 isalpha() 检测字符串是否只由字母组成(含汉字)
- 3 isdigit() 检测字符串是否只由数字组成
- 4 isspace() 检测是否为空白符
- 5 isupper() 是否为大写字母
- 6 islower() 是否为小写字母

```
1 >>> "sxt100".isalnum()
2 True
3 >>> "sxt尚学堂".isalpha()
4 True
5 >>> "234.3".isdigit()
6 False
7 >>> "23423".isdigit()
8 True
9 >>> "aB".isupper()
10 False
11 >>> "A".isupper()
12 True
13 >>> "\t\n".isspace()
14 True
```

实时效果反馈

1. 如下代码，打印结果是：

```
1 index = "我是高淇，一个程序员".find('高')  
2 print(index)
```

A 1

B 2

C 3

D 4

2. 如下代码，正确结果是：

```
1 str = "***s*x*t***".strip("**")  
2 print(str)
```

A sxt

B s*x*t**

C s*x*t

D **s*x*t

答案

1=>B 2=>C

字符串的格式化

format() 基本用法

基本语法是通过 `{}` 和 `:` 来代替以前的 `%`。

`format()` 函数可以接受不限个数的参数，位置可以不按顺序。

我们通过示例进行格式化的学习。

```
1 >>> a = "名字是:{0},年龄是: {1}"
2 >>> a.format("高淇",18)
3 '名字是:高淇,年龄是: 18'
4 >>> a.format("高希希",6)
5 '名字是:高希希,年龄是: 6'
6 >>> b = "名字是: {0}, 年龄是{1}。{0}是个好小伙"
7 >>> b.format("高淇",18)
8 '名字是: 高淇, 年龄是18。高淇是个好小伙'
9 >>> c = "名字是{name}, 年龄是{age}"
10 >>> c.format(age=19,name='高淇')
11 '名字是高淇, 年龄是19'
```

我们可以通过{索引}/{参数名}，直接映射参数值，实现对字符串的格式化，非常方便。

填充与对齐

- 1 填充常跟对齐一起使用
- 2 `^`、`<`、`>` 分别是居中、左对齐、右对齐，后面带宽度
- 3 `:` 号后面带填充的字符，只能是一个字符，不指定的话默认是用空格填充

```

1 >>> "{:*>8}".format("245")
2 '*****245'
3 >>> "我是{0},我喜欢数字{1:*^8}".format("高
  淇","666")
4 '我是高淇,我喜欢数字**666**'
```

数字格式化

浮点数通过 `f`，整数通过 `d` 进行需要的格式化。案例如下：

```

1 >>> a = "我是{0}，我的存款有{1:.2f}"
2 >>> a.format("高淇", 3888.234342)
3 '我是高淇，我的存款有3888.23'
```

其他格式，供大家参考：

数字	格式	输出	描述
3.1415926	{:.2f}	3.14	保留小数点后两位
3.1415926	{:+.2f}	3.14	带符号保留小数点后两位
2.71828	{:.0f}	3	不带小数
5	{:0>2d}	05	数字补零 (填充左边, 宽度为2)
5	{:x<4d}	5xxx	数字补x (填充右边, 宽度为4)
10	{:x<4d}	10xx	数字补x (填充右边, 宽度为4)
1000000	{:,}	1,000,000	以逗号分隔的数字格式
0.25	{:.2%}	25.00%	百分比格式
1000000000	{:.2e}	1.00E+09	指数记法
13	{:10d}	13	右对齐 (默认, 宽度为10)
13	{:<10d}	13	左对齐 (宽度为10)
13	{:^10d}	13	中间对齐 (宽度为10)

实时效果反馈

1. 如下代码，正确结果是：

```
1 c = "名字是{name}，年龄是{age}"  
2 d = c.format(age=19, name='高淇')  
3 print(d)
```

- ☐ A 名字是高淇，年龄是19
- ☐ B 名字是{高淇}，年龄是{19}
- ☐ C 名字是{name}，年龄是{age}
- ☐ D 名字是19，年龄是高淇

2. 如下代码，正确结果是：

```
1 a = "我是{0}，我的存款有{1:.2f}"  
2 b = a.format("高淇", 3888.234342)  
3 print(b)
```

- ☐ A 我是高淇，我的存款有3888
- ☐ B 我是{0}，我的存款有{1:.2f}
- ☐ C 我是高淇，我的存款有3888.23
- ☐ D 我是高淇，我的存款有3888.234342

答案

1=>A 2=>C

可变字符串



- 1 Python中，字符串属于不可变对象，不支持原地修改，如果需要修改其中的值，只能创建新的字符串对象。
- 2 确实需要原地修改字符串，可以使用io.StringIO对象或array模块

```
1 import io
2 s = "hello, sxt"
3 sio = io.StringIO(s)    #可变字符串
4 print(sio)
5 v1 = sio.getvalue()
6 print("v1:",v1)
7 char7 = sio.seek(7)     #指针知道索引7这个位置
8 sio.write("gaoqi")
9 v2 = sio.getvalue()
10 print("v2:",v2)
```

实时效果反馈

1. 如下代码，如下说法正确的是：

```
1 import io
2 s = "hello, sxt"
3 sio = io.StringIO(s)    #可变字符串
```

- ☐ A s修改为可变，sio是不可变
- ☐ B s仍然不可变，sio也是不可变
- ☐ C s修改为可变，sio也是可变
- ☐ D s仍然不可变，sio是可变的

答案

1=>D

类型转换总结

与C++、Java等高级程序设计语言一样，Python语言同样也支持数据类型转换。

类型转换	
int(x [,base])	将x转换为一个整数
long(x [,base])	将x转换为一个长整数
float(x)	将x转换到一个浮点数
complex(real[,imag])	创建一个复数
str(x)	将对象 x 转换为字符串
repr(x)	将对象 x 转换为表达式字符串
eval(str)	用来计算在字符串中的有效Python表达式,并返回一个对象
Complex(A)	将参数转换为复数型
tuple(s)	将序列 s 转换为一个元组
list(s)	将序列 s 转换为一个列表
set(s)	转换为可变集合
dict(d)	创建一个字典。d 必须是一个序列 (key,value)元组
frozenset(s)	转换为不可变集合
chr(x)	将一个整数转换为一个字符
unichr(x)	将一个整数转换为Unicode字符
ord(x)	将一个字符转换为它的整数值
hex(x)	将一个整数转换为一个十六进制字符串
oct(x)	将一个整数转换为一个八进制字符串

```

1  #类型转换
2
3  #转换为int
4  print('int()默认情况下为: ', int())
5  print('str字符型转换为int: ', int('010'))
6  print('float浮点型转换为int: ', int(234.23))
7  #十进制数10, 对应的2进制, 8进制, 10进制, 16进制分别是: 1010,12,10,0xa
8  print('int(\'0xa\', 16) = ', int('0xa', 16))

```

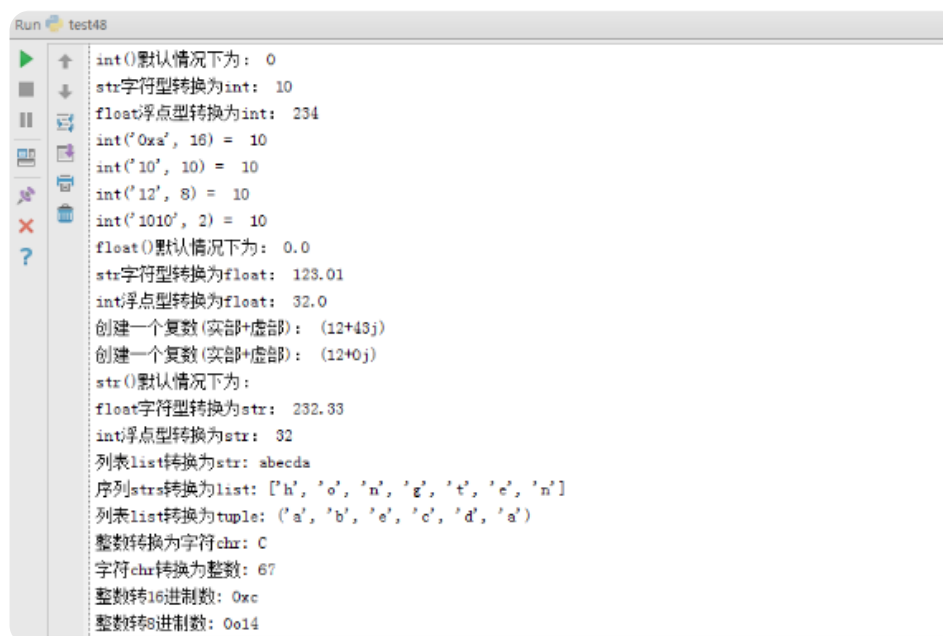
```
9 print('int(\'10\', 10) = ', int('10', 10))
10 print('int(\'12\', 8) = ', int('12', 8))
11 print('int(\'1010\', 2) = ', int('1010', 2))
12
13 #转换为float
14 print('float()默认情况下为: ', float())
15 print('str字符型转换为float: ',
16       float('123.01'))
17
18 #转换为complex
19 print('创建一个复数(实部+虚部): ', complex(12,
20       43))
21
22 print('创建一个复数(实部+虚部): ', complex(12))
23
24 #转换为str字符串
25 print('str()默认情况下为: ', str())
26 print('float型转换为str: ', str(232.33))
27 print('int转换为str: ', str(32))
28
29 lists = ['a', 'b', 'e', 'c', 'd', 'a']
30 print('列表list转换为str:', ''.join(lists))
31
32
33 #转换为list
34 strs = 'hongten'
35 print('序列strs转换为list:', list(strs))
36
37
38 #转换为tuple
39 print('列表list转换为tuple:', tuple(lists))
40
41
42 #字符和整数之间的转换
43 print('整数转换为字符chr:', chr(67))
44 print('字符chr转换为整数:', ord('C'))
```

```

39
40 print('整数转16进制数:', hex(12))
41 print('整数转8进制数:', oct(12))

```

运行效果:



```

Run test48
int()默认情况下为: 0
str字符串转换为int: 10
float浮点型转换为int: 234
int('0xa', 16) = 10
int('10', 10) = 10
int('12', 8) = 10
int('1010', 2) = 10
float()默认情况下为: 0.0
str字符串转换为float: 123.01
int浮点型转换为float: 32.0
创建一个复数(实部+虚部): (12+43j)
创建一个复数(实部+虚部): (12+0j)
str()默认情况下为:
float字符串转换为str: 232.33
int浮点型转换为str: 32
列表list转换为str: abcda
序列strs转换为list: ['h', 'o', 'n', 'g', 't', 'e', 'n']
列表list转换为tuple: ('a', 'b', 'e', 'c', 'd', 'a')
整数转换为字符chr: C
字符chr转换为整数: 67
整数转16进制数: 0xc
整数转8进制数: 0o14

```

实时效果反馈

1. 转整数、转浮点数、转字符串，分别是哪几个方法：

- A
- B
- C
- D

答案

1=>A